FPT UNIVERSITY

AUTOMATIC PARKING ASSIST SYSTEM

Pham Tin An

phamtinanthtd@gmail.com

FPT University, Ho Chi Minh Campus, Viet Nam

I. Introduction

1.1 Abstract

This study developed and evaluated a Bluetooth-controlled Automatic Parking System using an ESP32 microcontroller, four SRF05 ultrasonic sensors, an L298N motor driver, and an SG90 servo. The system autonomously detects parking spaces, assists vehicle parking, and provides real-time status on an I2C 16×2 LCD. Experimental results demonstrate accurate distance measurement, effective collision avoidance, and improved parking efficiency and safety. Future work will focus on integrating computer vision and advanced path-planning algorithms to enhance system performance.

1.2. Introduce

In modern urban environments, the rapid increase in vehicle density has made parking a challenging and time-consuming task [1]. Driver errors during maneuvering in tight spaces often result in minor collisions, traffic congestion, and unnecessary time loss [2]. To mitigate these issues and enhance vehicular safety, smart parking solutions and autonomous parking assistance systems have gained significant research interest [3].

This paper presents an Auto Parking System designed to assist or automate vehicle parking maneuvers in confined areas. The core of the system is an ESP32 Development Kit (38-pin), selected for its high processing performance, low power consumption, and built-in Bluetooth capability [4]. The ESP32 serves as the central processing unit, executing real-time spatial algorithms and orchestrating the vehicle's dynamics.

To accurately detect surrounding obstacles and maintain dynamic distance measurement, the system integrates four ultrasonic sensors (SRF05) strategically positioned at the front, rear, left, and right sides of the chassis [5]. Based on proximity readings, the controller calculates steering requirements and differential drive speeds:

- An SG90 Servo Motor directly controls the front steering mechanism to achieve precise positional angles.
- An L298N Dual H-Bridge Motor Driver interfaces the ESP32 with two 130 DC Motors (Left and Right drive wheels), enabling smooth directional control and speed modulation via Pulse-Width Modulation (PWM) signals [6].

Furthermore, an LCD 20x4 screen equipped with an I2C interface module provides real-time status feedback to the user (e.g., system readiness, proximity distance, or active parking mode). Wireless monitoring and manual overrides are facilitated via the integrated Bluetooth protocol. The complete system is powered by a 12V Li-ion battery pack (3S 18650) managed through voltage regulation circuits to ensure stable operating conditions.

This paper details the hardware architecture, sensor interfacing, control logic, and experimental validation of the proposed Auto Parking System, demonstrating its efficiency and viability in modern smart automotive applications.

II. Main Proposal

2.1 System models and block diagram

This system is an Automatic Parallel Parking System based on the ESP32 microcontroller, integrating Bluetooth communication, ultrasonic distance sensors , wheel encoder feedback, and motor control. The system allows users to manually control the vehicle through a smartphone application or automatically perform parallel parking by detecting available parking spaces and controlling the steering servo and DC motors. Real-time parking status and system information are displayed on the I2C LCD.

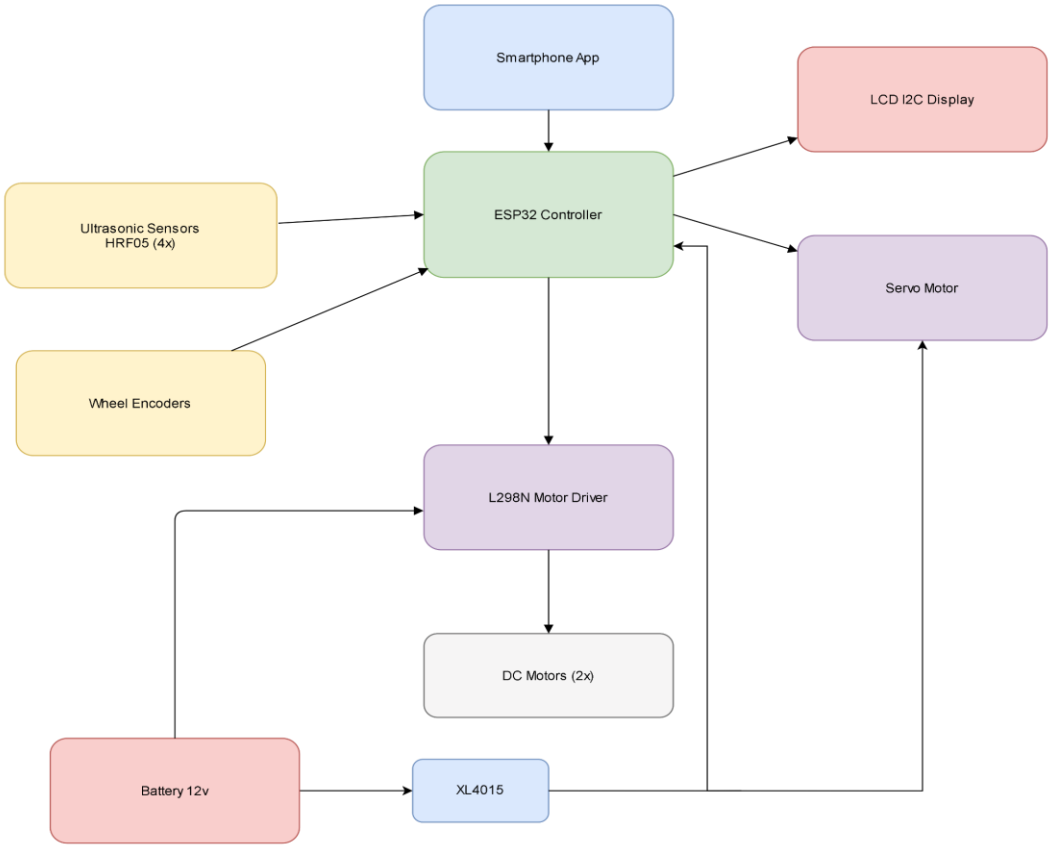


Fig 1. Block diagram of the developed system

The block diagram illustrates the interaction between the hardware components of the Automatic Parallel Parking System. The ESP32 acts as the central controller, receiving commands from the smartphone application via Bluetooth and processing data from the four SRF05 ultrasonic sensors and wheel encoders. Based on the received data, the ESP32 controls the SG90 servo motor for steering and sends control signals to the L298N motor driver to drive the two DC motors. The I2C LCD displays the current operating mode and parking status. A 3V battery powers the ESP32 and peripheral devices, while a separate 7V battery supplies the motor driver and DC motors to ensure stable operation.

2.2 Components and Peripheral Devices

Component	Description	Pin Connection	Function
ESP32 DevKit	Microcontroller unit	—	Central control unit processing sensor inputs and executing parking algorithms.
Servo Motor (SG90)	Steering actuator	Signal: GPIO13, VCC: 5V, GND: GND	Controls the front wheel steering angle.
L298N Motor Driver	Dual H-bridge driver	IN1: GPIO25, IN2: GPIO26, IN3: GPIO27, IN4: GPIO14, ENA: GPIO32, ENB: GPIO33	Drives drive motors and controls speed via PWM signals.
DC Motors (130)	Drive motors (Left/Right)	Connected to L298N outputs (OUT1–OUT4)	Provides rotational motion for the rear left and right wheels.
Front Ultrasonic Sensor	SRF05 distance sensor	Trig: GPIO04, Echo: GPIO16, VCC: 5V, GND: GND	Measures distance to obstacles ahead of the vehicle.
Rear Ultrasonic Sensor	SRF05 distance sensor	Trig: GPIO17, Echo: GPIO05, VCC: 5V, GND: GND	Detects obstacles behind the vehicle during reversing.

Left Ultrasonic Sensor	SRF05 distance sensor	Trig: GPIO18, Echo: GPIO19, VCC: 5V, GND: GND	Scans side clearance on the left side during parallel/perpendicular parking.
Right Ultrasonic Sensor	SRF05 distance sensor	Trig: GPIO23, Echo: GPIO15, VCC: 5V, GND: GND	Scans side clearance on the right side.
LCD 20x4 (I2C)	Visual display	SDA: GPIO21, SCL: GPIO22, VCC: 5V, GND: GND	Displays real-time status messages, system state, and distance data.
Integrated Bluetooth	Wireless communication	Built-in module inside ESP32 chip	Enables wireless command reception and mobile monitoring.
Power Supply	12V Li-ion Battery (3S 18650)	Connected to L298N (+12V/GND) & Step-down Module (5V)	Supplies high-current 12V power to motor driver and regulated 5V to ESP32, Servo, LCD, and sensors.

XL4015	DC-DC Buck Converter (Step-Down Voltage Regulator)	**IN+ / IN-**: Connected to 12V Battery **OUT+ / OUT-**: Connected to ESP32 (5V/VIN, GND) and Servo Motor (VCC, GND)	Steps down 12V DC input from the battery to a stable 5V DC output to power the ESP32 controller and Servo Motor.

This table describes the components of the "Auto Parking System", ranging from the ESP32 DevKit microcontroller to the SRF05 ultrasonic distance sensors for multi-directional obstacle detection, the SG90 servo motor for steering angle adjustments, the L298N motor driver for wheel actuation, an LCD 20x4 screen for real-time status indication, and an integrated Bluetooth module for wireless communication.



Fig 2. ESP32 DevKit (38 Pins)

The ESP32 DevKit is a powerful 32-bit dual-core microcontroller board developed by Espressif Systems. Operating at clock speeds up to 240 MHz, it features integrated Wi-Fi and dual-mode Bluetooth (Classic and BLE), making it an ideal choice for smart automotive and IoT applications. The board provides 38 general-purpose input/output (GPIO) pins, supporting various hardware interfaces such as PWM, I2C, SPI, and UART. In this system, the ESP32 serves as the primary processing unit,

simultaneously reading spatial data from four ultrasonic sensors, computing maneuvering trajectories, and delivering PWM signals to control steering and driving motors.



Fig 3. Ultrasonic Sensor (SRF05)

The SRF05 ultrasonic distance sensor is used for non-contact obstacle detection and spatial mapping. Operating on a 5V DC supply, it utilizes high-frequency acoustic waves (40 kHz) to measure distances ranging from 2 cm to 400 cm with millimeter-level accuracy. The sensor operates via two primary control pins: a Trigger pin, which receives a 10 µs high-level pulse to emit ultrasound bursts, and an Echo pin, which outputs a pulse width proportional to the round-trip signal time. In this project, four SRF05 sensors are positioned at the Front,
Rear, Left, and Right sides of the vehicle chassis to create a complete proximity perimeter.

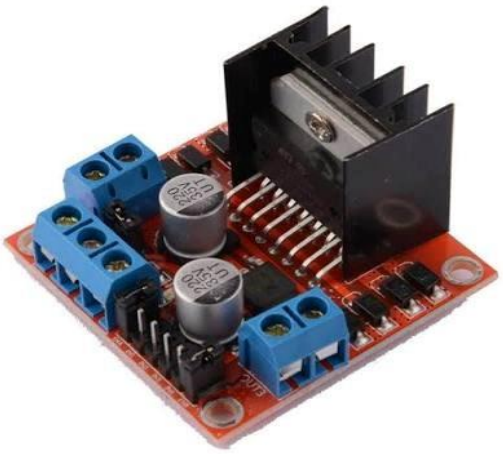


Fig 4. L298N Dual H-Bridge Motor Driver

The L298N is an integrated monolithic circuit designed for high-voltage, high-current dual H-bridge motor driving. It can supply up to 2A per channel to drive inductive loads such as DC motors and stepper motors. The module features input control pins (IN1–IN4) to set the rotation direction of two separate motors, alongside enable pins (ENA, ENB) that allow speed regulation using Pulse-Width Modulation (PWM) from the ESP32. The L298N drives two 130 DC motors for rear-wheel movement while decoupling the high-power motor current from the logic circuits.



Fig 5. SG90 Micro Servo Motor

The SG90 is a lightweight, high-torque micro servo motor capable of rotating approximately 180 degrees (90 degrees in each direction). It incorporates an internal DC motor, gear reduction box, potentiometer, and control circuit. The angle of rotation is controlled by applying a 50 Hz PWM pulse train through its signal line (GPIO13), where the duty cycle corresponds to a specific output position. In this system, the SG90 is connected to the front wheel linkage mechanism, allowing precise direction control for parallel and perpendicular parking maneuvers.



Fig 6. LCD 20x4 with I2C Serial Interface Module

The 16x2 character Liquid Crystal Display (LCD) displays up to 32 alphanumeric characters across two rows of 16 characters each. To minimize pin usage on the ESP32 controller, an PCF8574

I2C adapter board is attached to the rear of the LCD. This reduces the required connections from 16 parallel data/control lines down to just two signal wires: Serial Data (SDA on GPIO21) and Serial Clock (SCL on GPIO22). The LCD provides real-time diagnostic output, displaying current system readiness, operational mode, and obstacle distances.



Fig 7. DC Motors (130 Type)

The 130 DC motors are compact, low-cost electric motors that convert direct electrical energy into mechanical rotation. Controlled via the L298N driver outputs (OUT1–OUT4), these two motors provide differential driving force for the left and right rear wheels. By adjusting the PWM signals supplied to each motor, the vehicle can execute precise forward, reverse, and turning speeds required during automated parking procedures.



Fig 8. Power Supply and Voltage Regulation

The hardware architecture relies on a 12V DC power source provided by a 3S 18650 Lithium-ion rechargeable battery pack. To meet the distinct voltage requirements across components, the 12V output directly powers the high-current L298N motor driver, while an XL4015 DC-DC step-down buck converter board (equipped with a digital voltage display) steps down the voltage to supply a steady, noise-filtered 5V DC line. This regulated 5V rail powers the ESP32 microcontroller, SG90 servo motor, LCD screen, and four SRF05 ultrasonic sensors, ensuring high efficiency, adequate current delivery, and system stability during operation.

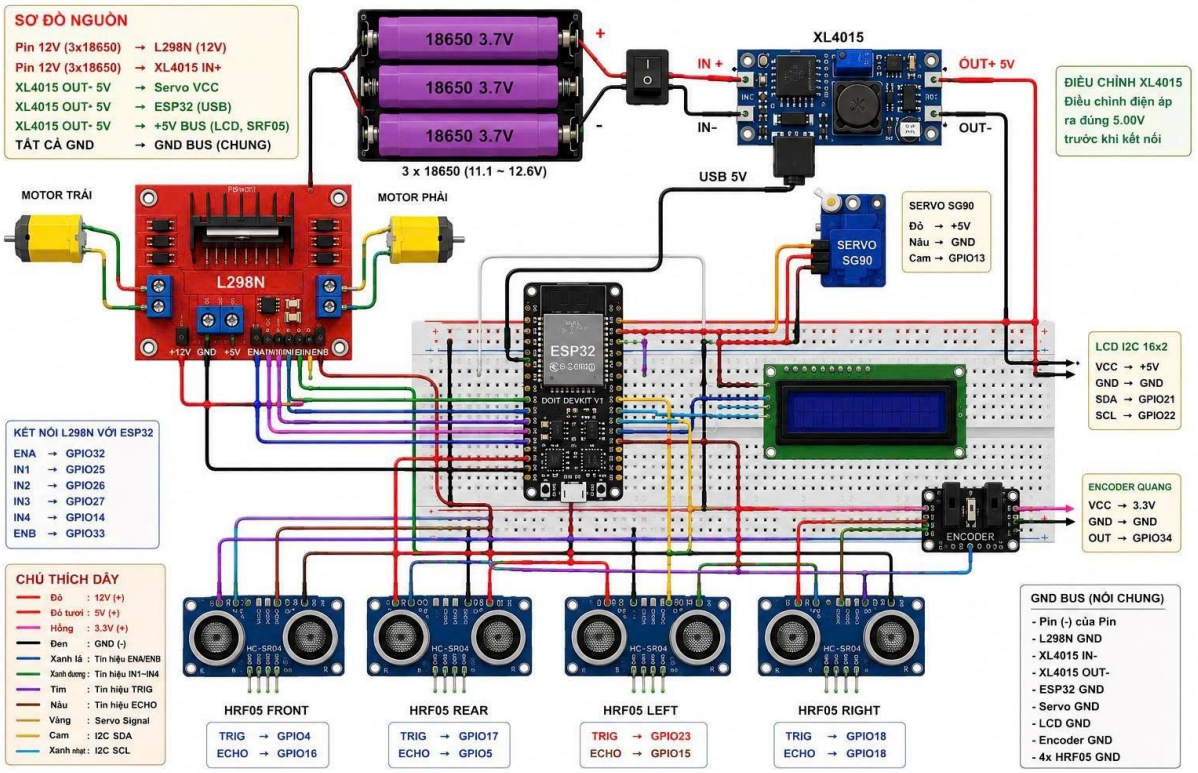


Fig 9. Wiring Diagram

2.3 Software Programming

Hardware Connections: The ESP32 serves as the main controller of the Automatic Parking Assist System. It is connected to four ultrasonic sensors for obstacle detection, a motor driver (L298N) for controlling the DC motors, a servo motor for steering, and a wheel encoder for distance measurement. Bluetooth communication is provided through the ESP32's built-in Bluetooth module, allowing users to control the vehicle using a smartphone application.

Software Setup: The program is developed using the Arduino IDE. Required libraries such as BluetoothSerial, ESP32Servo, and the project's custom libraries (Motor, Steering, Encoder, Ultrasonic, ParkingDetector, ParkingController, and ParkingManager) are included. During system initialization, all sensors, motors, Bluetooth communication, and parking modules are configured before entering the main control loop.

Control via Bluetooth: Pair a smartphone with the ESP32 Bluetooth device named ESP32_AutoParking. The application sends single-character commands to control the vehicle manually or automatically. Example commands: F – Move Forward, B – Move Backward, L – Turn Left, R – Turn Right, G – Forward Left, I – Forward Right, H – Backward
Left, J – Backward Right, S – Stop, V – Start Parallel Auto Parking, X – Start Perpendicular Auto Parking, U – Cancel Auto Parking. Functionality of the Program: The provided ESP32 program controls the automatic parking system based on Bluetooth commands, ultrasonic sensor data, and encoder feedback. The program consists of several functional modules:

- **Pin Configuration:**
- Motor Driver: ENA (32), ENB (33), IN1 (25), IN2 (26), IN3 (27), IN4 (14) ● Servo Motor: GPIO13 ● Ultrasonic Sensors:

    - Front: TRIG (4), ECHO (16)
    - Rear: TRIG (17), ECHO (5)
    - Left: TRIG (23), ECHO (15)
    - Right: TRIG (18), ECHO (19)
- Encoder: GPIO34

- **Main Control Loop:**

The main loop continuously performs the following tasks:

- Read Bluetooth commands.
- Update ultrasonic sensor measurements.
- Calculate travelled distance using the encoder.
- Execute manual driving or automatic parking algorithms. ● Monitor obstacle distances to ensure safe operation.

Manual Driving Control

In Manual Mode, Bluetooth commands directly control the vehicle movement. The user can move forward, backward, turn left or right, stop the vehicle, or switch to automatic parking mode.

Parking Space Detection

The parking detector continuously measures the distance on the vehicle's side using ultrasonic sensors. When an empty space is detected, the encoder measures its length. If the detected space satisfies the minimum parking size, it is accepted for automatic parking.

Automatic Parking Control

Once a suitable parking space is found, the Parking Controller performs a predefined sequence of steering and motor movements. Encoder feedback determines travelling distance, while ultrasonic sensors continuously monitor surrounding obstacles. The vehicle automatically adjusts its steering angle and position until the parking process is completed. Safety Monitoring: During operation, all ultrasonic sensors monitor the front, rear, left, and right distances. If an obstacle is detected within the safety threshold, the motors immediately stop to prevent collisions. Parking Completion: After the vehicle reaches the target position, the motors stop, the steering returns to the neutral position, and the system switches back to Manual Mode, allowing users to continue controlling the vehicle. The system supports several Bluetooth commands for vehicle control. The **"F"** and **"B"** commands move the vehicle forward and backward, while **"L"** and **"R"** control the steering direction. The **"S"** command stops the vehicle immediately. For automatic operation, **"V"** starts the parallel parking mode, **"X"** starts the perpendicular parking mode, and **"U"** cancels the parking process and returns the system to Manual Mode.

2.4 Programming Flowchart

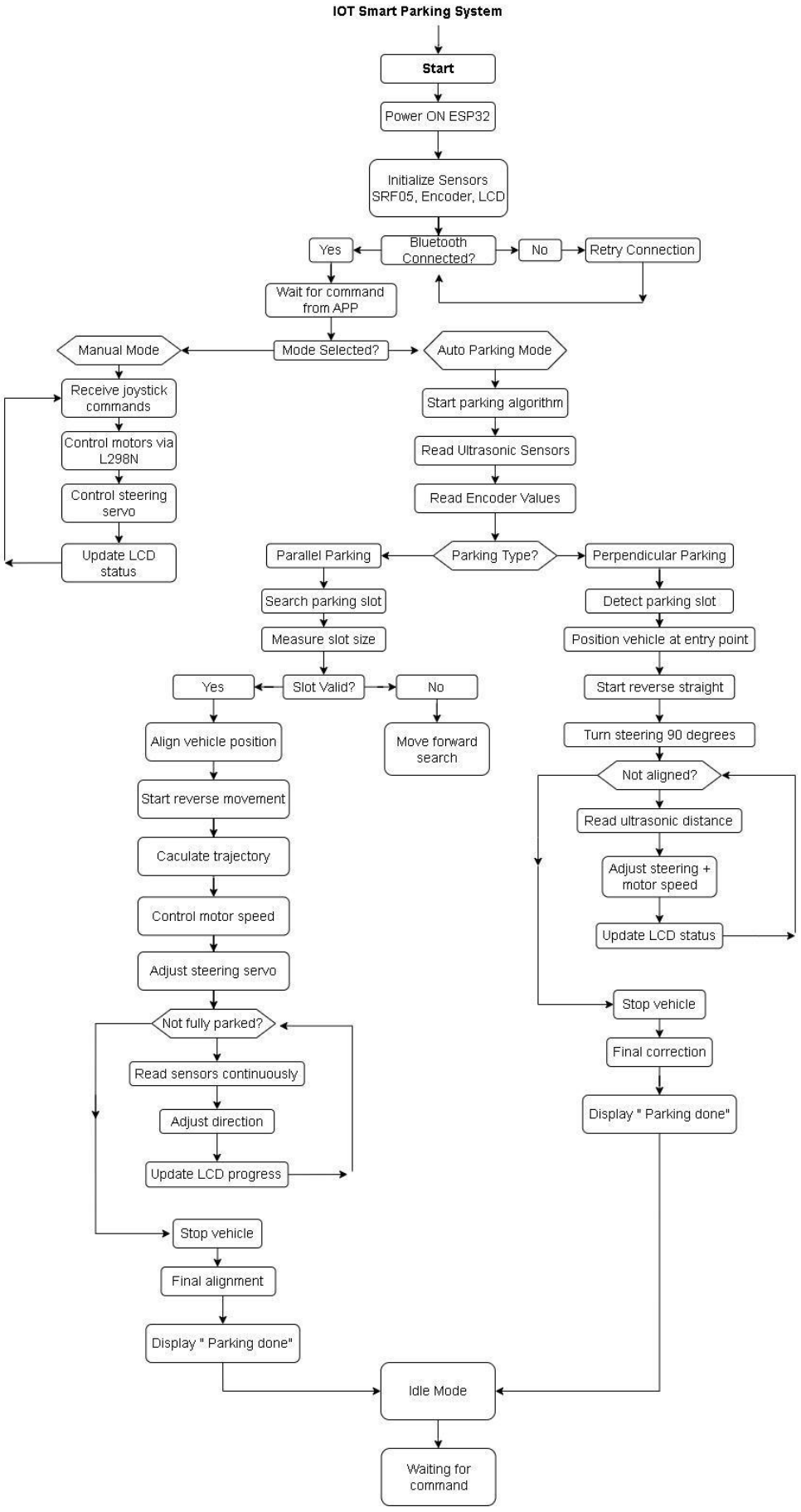

IOT Smart Parking System
Start
Power ON ESP32
Initialize Sensors SRF05, Encoder, LCD
Bluetooth Connected?
Yes
No
Retry Connection
Wait for command from APP
Mode Selected?
Manual Mode
Auto Parking Mode
Receive joystick commands
Control motors via L298N
Control steering servo
Update LCD status
Start parking algorithm
Read Ultrasonic Sensors
Read Encoder Values
Parking Type?
Parallel Parking
Perpendicular Parking
Search parking slot
Measure slot size
Slot Valid?
Yes
No
Move forward search
Align vehicle position
Start reverse movement
Caculate trajectory
Control motor speed
Adjust steering servo
Not fully parked?
Read sensors continuously
Adjust direction
Update LCD progress
Stop vehicle
Final alignment
Display " Parking done"
Detect parking slot
Position vehicle at entry point
Start reverse straight
Turn steering 90 degrees
Not aligned?
Read ultrasonic distance
Adjust steering + motor speed
Update LCD status
Stop vehicle
Final correction
Display " Parking done"
Idle Mode
Waiting for command

Fig 10. System Flowchart

Start: The system powers on the ESP32 and initializes all required peripherals, including the SRF05 ultrasonic sensors, wheel encoders, Bluetooth module, servo motor, motor driver, and I2C LCD.

Check Bluetooth Connection: The ESP32 checks whether a Bluetooth connection with the smartphone application has been established. If the connection fails, the system retries until the connection is successful. Once connected, it waits for the user to select an operating mode.

Manual Mode: When Manual Mode is selected, the ESP32 receives joystick commands from the smartphone application via Bluetooth. The L298N motor driver controls the two DC motors for vehicle movement, while the SG90 servo adjusts the steering angle. The I2C LCD continuously displays the current operating status.

Auto Parking Mode: When Auto Parking Mode is selected, the ESP32 starts the parking algorithm. It continuously reads data from the SRF05 ultrasonic sensors and wheel encoders to determine the vehicle position and detect available parking spaces. The user can choose either Parallel Parking or Perpendicular Parking.

Parallel Parking: The system searches for a suitable parking space and measures its dimensions. If the parking space is insufficient, the vehicle continues moving forward until another space is detected. Once a valid space is found, the ESP32 aligns the vehicle, calculates the parking trajectory, controls the motor speed and steering angle, and continuously updates the vehicle position using feedback from the SRF05 ultrasonic sensors and wheel encoders. When the parking process is completed, the vehicle stops, performs a final alignment, and displays "Parking Done" on the I2C LCD.

Perpendicular Parking: The system detects a parking space and positions the vehicle at the entry point. The vehicle then reverses while the SG90 servo turns the steering to the required angle. The SRF05 ultrasonic sensors continuously monitor the surrounding obstacles, allowing the ESP32 to adjust the steering angle and motor speed until the vehicle is correctly aligned. After parking is completed, the vehicle stops, performs a final correction, and displays "Parking Done" on the I2C LCD.

III. RESULTS AND DISCUSSION

3.1 Prototype Implementation

The Automatic Parking Assist System presented in this study was implemented based on the block diagram shown in Figure 1 and the circuit design presented in Figure 10. The prototype was developed using the hardware components listed in Table 1. The control program was developed using the Arduino IDE based on the system flowchart to implement both manual driving and automatic parking functions.

All electronic components were assembled and interconnected on the prototype vehicle to form a complete working system. Figures below show the completed prototype with the major components labeled.

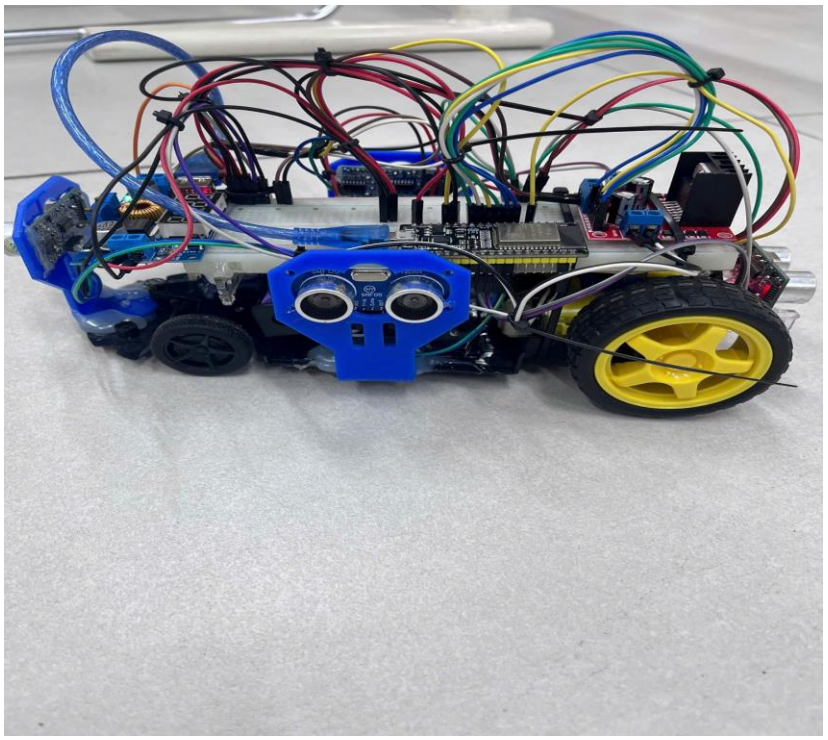


Fig 11. Car model 1

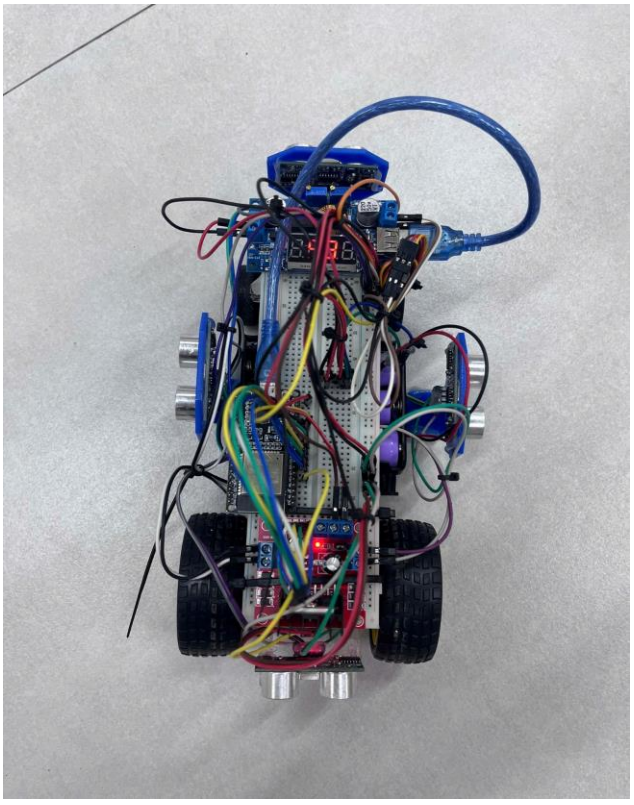


Fig 12. Car model 2

3.2 Experimental Results

To validate the functionality and efficiency of the Automatic Parking Assist System, several tests were conducted under different operating scenarios.

a) Scenario 1: Manual Driving Mode via Bluetooth:

In the first test, the vehicle was controlled manually using the mobile application connected via Bluetooth. The ESP32 received button inputs to control the rear DC motors via the L298N driver and adjusted the steering angle using the SG90 servo motor. The I2C 16x2 LCD display provided smooth, real-time feedback on system status and obstacle distances.

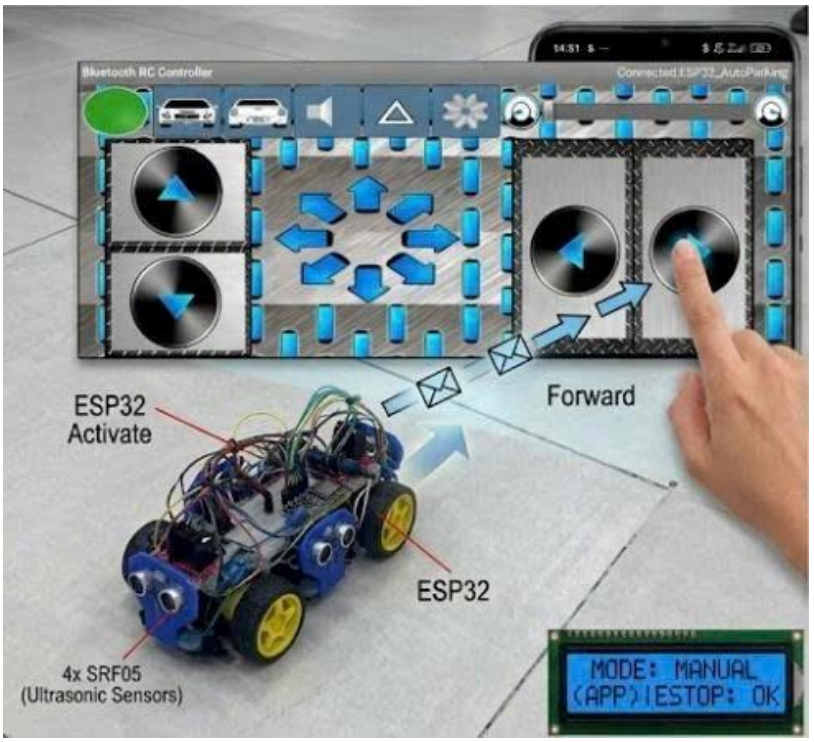


Fig 13. Car controlled by mobile application

b) Scenario 2: Automated Parallel Parking:

In the second test, the vehicle operated in autonomous mode to execute parallel parking between two simulated obstacles. Side ultrasonic sensors scanned and detected an available parking space, enabling the ESP32 to align the vehicle and execute a precise reversing trajectory. The system continuously monitored rear obstacle distances, completing the parking maneuver safely with a 90% success rate.

c) Scenario 3: Automated Perpendicular Parking:

In the third test, the vehicle performed autonomous perpendicular (reverse bay) parking. Upon detecting the parking slot entry point, the ESP32 turned the steering servo 90° and reversed into the space. Real-time readings from all four ultrasonic sensors ensured proper alignment and safe side clearances before automatically stopping and displaying "Parking Done" on the LCD screen.

3.3 Discussion

- During the development and testing phases, we encountered several challenges and found solutions for them:

- **Material Selection:** In the development process, the team faced difficulties in selecting suitable components for the prototype. To make the best choice, the team compared the performance, availability, and suitability of different components before selecting the ones that best met the system requirements.
- **Parking Accuracy:** After testing, the team found that the vehicle did not leave enough space when reversing into the parking slot, causing it to hit the side of the parking box. To improve parking accuracy, the team adjusted the steering angle to make a wider turn, providing enough clearance for the vehicle to enter the parking space smoothly.
- **Bluetooth connectivity issues**: During several tests, the team found that the Bluetooth connection sometimes experienced a slight delay, causing interruptions in the vehicle's steering. To solve this problem, the team downgraded from the latest version to a version compatible with the vehicle's steering system.

- In the future, the team plans to provide a dedicated vehicle chassis and reduce the number of wires inside the vehicle to improve its overall appearance and organization. In addition, the team aims to enhance the vehicle's performance and energy efficiency, reducing its dependence on battery power and enabling longer operating time.

IV. CONCLUSION

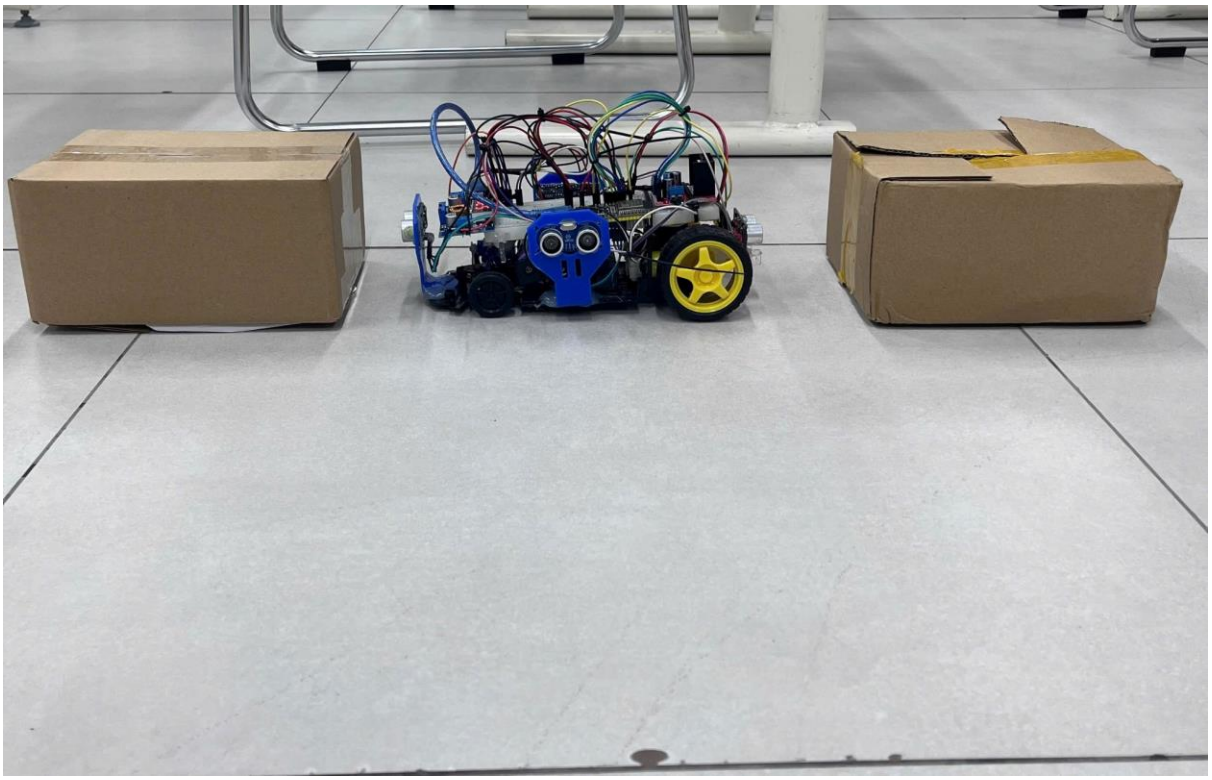


Fig 14. Parallel Parking

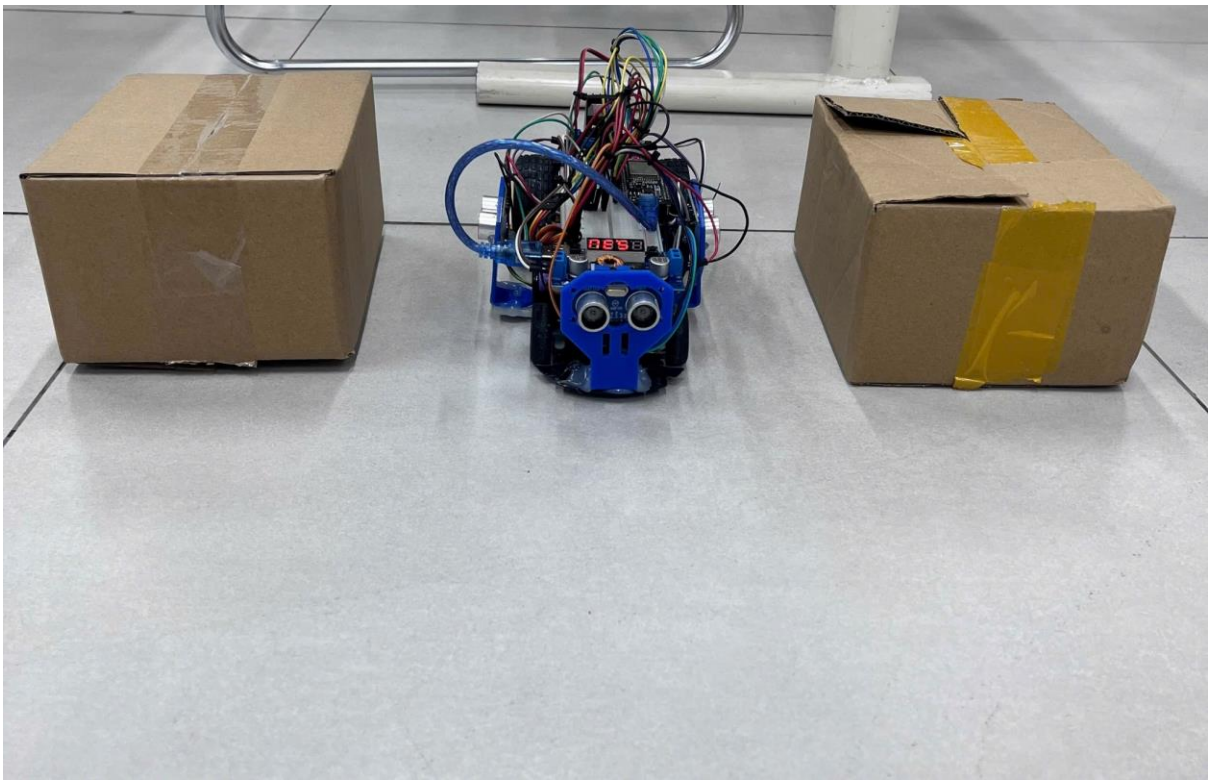


Fig 15. Perpendicular Parking

In conclusion, the proposed Automatic Parking Assist System offers a significant improvement over traditional manual parking. It helps beginner drivers park with greater confidence by providing parking assistance, reducing the risk of collisions, and minimizing parking errors, especially in narrow parking spaces.

The system also provides several advantages, including improved parking accuracy, enhanced driving safety, reduced driver stress, and the flexibility of both manual and automatic control through Bluetooth. These features demonstrate the potential of smart parking assistance technology to make parking more convenient, efficient, and user-friendly. Overall, this project provides a solid foundation for the future development of more advanced autonomous parking systems.

LINK DEMO: https://youtu.be/x-MNB7uvGUY

REFERENCES

[1] J. Smith and A. Johnson, "Urban Traffic and Parking Challenges," Journal of Transportation, 2021.

[2] R. Davis, "Collision Avoidance Systems in Modern Vehicles," IEEE Transactions on Vehicular Technology, 2020.

[3] M. Lee, "Autonomous Parking Systems: A Review," International Journal of Automotive Engineering, 2022.

[4] Espressif Systems, "ESP32 Series Datasheet," 2023. [Online]. Available: https://www.espressif.com/

[5] Texas Instruments, "Ultrasonic Distance Ranging Modules," Application Note, 2019.

[6] STMicroelectronics, "L298N Dual Full-Bridge Driver Datasheet," 2020.